

# Filmception Project Report

## 1. Introduction

Filmception represents an innovative fusion of artificial intelligence, natural language processing, and multilingual capabilities designed to transform how users interact with movie content. As an AI-powered multilingual movie summary translator and genre classifier, this project demonstrates the practical application of advanced machine learning techniques to solve real-world content processing challenges.

The system offers three core functionalities that work together seamlessly:

1. **Movie Summary Processing:** Users can input any movie summary text for analysis and transformation
2. **Multilingual Audio Conversion:** The system translates summaries into multiple languages (Arabic, Urdu, and Korean) and converts them to natural-sounding audio
3. **Intelligent Genre Classification:** Using a state-of-the-art transformer model, the system accurately predicts movie genres from summary text

Built upon the comprehensive CMU Movie Summary dataset, Filmception implements a complete pipeline from raw data processing to an intuitive user interface. This report details our methodological approach, the key decisions made throughout development, and the reasoning behind our technical choices.

## 2. Data Preprocessing and Cleaning

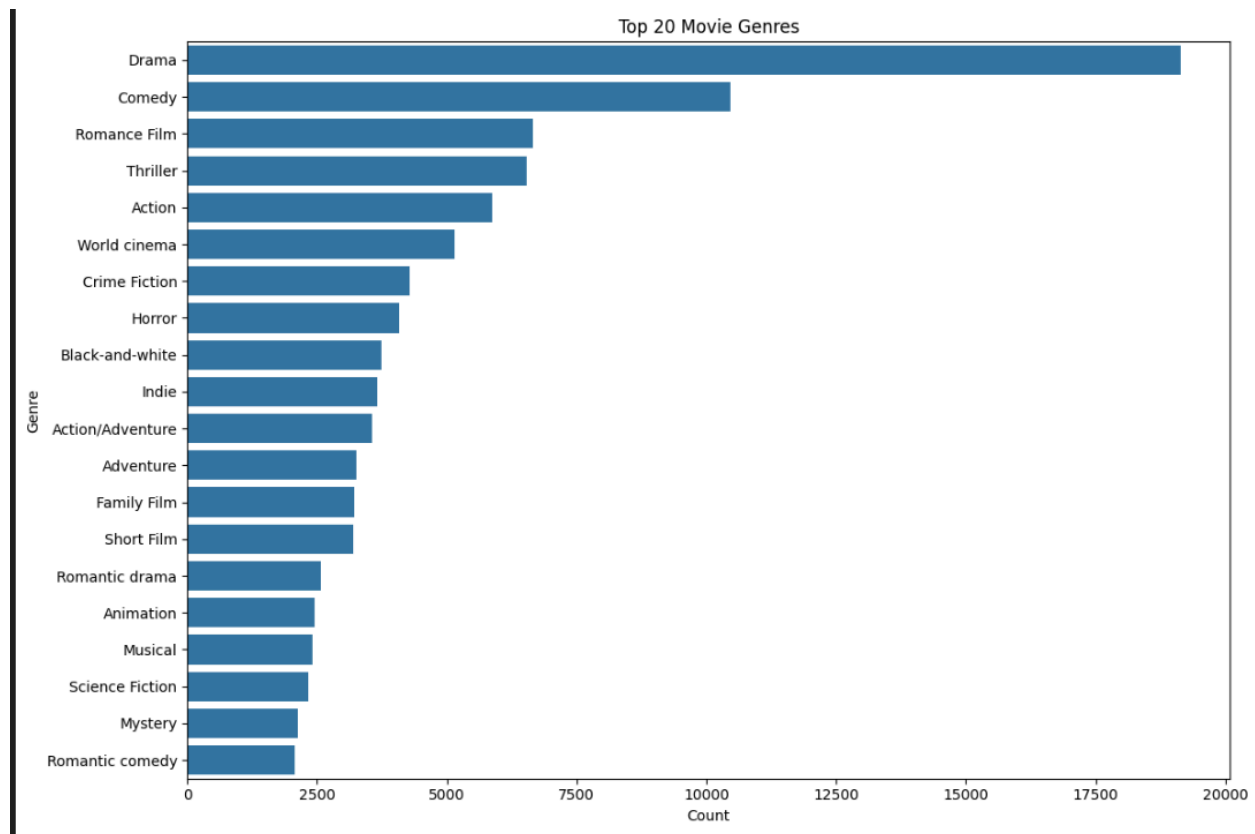
### Data Loading and Exploration

The foundation of Filmception lies in its data preprocessing pipeline, which begins with the careful loading and exploration of the CMU Movie Summary dataset. This dataset provides rich information about movies, including both structured metadata and unstructured text summaries.

**Methodology:** - We began by loading two primary data files: `movie.metadata.tsv` containing structured movie information and `plot_summaries.txt` containing textual summaries - These datasets were merged on `movie_id` to create a unified dataset connecting summaries with their corresponding metadata - Exploratory data analysis revealed important dataset characteristics, including the distribution of genres (some like Drama being much more common than others) and the varying lengths of movie summaries

**Decisions and Reasoning:** - The decision to merge datasets early in the pipeline was made to maintain data integrity and ensure proper alignment between summaries and their corresponding genre information - Understanding genre distribution was critical for addressing potential class imbalance issues later in the modeling phase - Analyzing

summary length distributions helped establish appropriate parameters for the text processing pipeline, including tokenization limits



## Text Preprocessing

Movie summaries required systematic preprocessing to enhance both the translation quality and genre prediction accuracy. Our approach incorporated standard NLP techniques tailored to our specific requirements.

### Methodology:

- 1. Lowercase Conversion:** All text was converted to lowercase to standardize case treatment
- 2. Special Character Removal:** Non-alphabetic characters were removed using regular expressions
- 3. Tokenization:** Text was split into individual tokens using NLTK's `word_tokenize`
- 4. Stopword Removal:** Common English stopwords were filtered out
- 5. Stemming:** The Porter stemming algorithm was applied to reduce words to their base forms
- 6. Reconstruction:** Processed tokens were rejoined to form cleaned summaries

**Decisions and Reasoning:** - Each preprocessing step addressed specific data quality concerns: - Lowercase conversion eliminated inconsistencies in capitalization that could artificially inflate the vocabulary size - Special character removal focused the model on meaningful textual content rather than punctuation or formatting - Stopword removal eliminated high-frequency, low-information words that typically don't contribute to genre differentiation - Stemming reduced vocabulary sparsity by consolidating different forms of the same word

- For the BERT-based genre prediction model, we maintained a separate, lighter preprocessing approach since transformer models like BERT benefit from preserving more of the original text structure and have their own specialized tokenization processes
- This dual preprocessing approach allowed us to optimize each pipeline component according to its specific requirements, rather than forcing a one-size-fits-all solution

### 3. Multilingual Translation

#### Translation Strategy

A key innovation in Filmception is its ability to transform movie summaries into multiple languages, making content accessible to diverse linguistic audiences. Our translation implementation required careful consideration of language selection, API integration, and performance optimization.

**Methodology:** - We implemented translation using the Google Translate API through the `googletrans` Python library - Movie summaries were translated into three target languages: Arabic, Urdu, and Korean - To manage API rate limits and ensure system stability, translations were processed in batches with appropriate delays - Each translation was stored both in memory and in language-specific directories following a consistent naming convention - Error handling mechanisms were implemented to gracefully manage translation failures

**Decisions and Reasoning: - Language Selection:** The three target languages (Arabic, Urdu, and Korean) were deliberately chosen to demonstrate the system's versatility across diverse linguistic features: - Arabic: A right-to-left script with complex morphology - Urdu: Another right-to-left script with cultural and linguistic connections to Arabic and Hindi - Korean: A language with a unique alphabet (Hangul) and sentence structure very different from English

- **API Selection:** Google Translate was chosen after evaluating several translation services because it offered:
  - Superior quality across our target languages
  - Robust API documentation and support
  - Reasonable rate limits for our development purposes

- ```

"7275095": {
  "original": "Mexican funnyman, El Clavillazo , is in love with a seamstress named Beatriz , and also hangs out with a variety of odd characters, including a",
  "translations": {
    "Arabic": "مفوض ومرحب عظمي. في هذه الأثناء ، في اللغة العربية ، يشغل عالم محبون يدعى Beatriz في حالة مع حياطة تدعى El Clavillazo ، Mexican Funnyman ،",
    "Urdu": "اس میں طرح طرح کے عجیب و غریب کرداروں کے ساتھ بھی بھانسی دی گئی ہے ، جس میں ایک کمزور بوائے اور دینی مریض بھی شامل ہے۔ دریں اثنا ، فریبی فلمی ہیں۔",
    "Korean": "멕시코의 재밌는 사람인 엘 클라빌라조 (El Clavillazo)는 Beatriz라는 재봉사와 사람에 빠져있으며 뉴스 보이 및 정신 환자를 포함한 다양한 이상한 캐릭터와 어울립니다. 한편, 인근"
  }
}

```

- **Audio Quality Optimization:** We configured gTTS to balance audio quality with file size and generation speed:
  - Standard speech rate was used to ensure clarity and comprehension
  - MP3 format was selected for its universal browser compatibility and good compression ratio
  - Appropriate error handling was implemented to manage potential issues with unusual characters or very long texts
- **Directory Structure Design:** The hierarchical organization by language and movie ID facilitated efficient retrieval and maintenance, with each audio file named according to its source movie ID for consistent referencing



## 5. Genre Prediction Model Development

### Model Selection and Architecture

The core intelligence of Filmception lies in its ability to predict movie genres from textual summaries. This multi-label classification task required a sophisticated approach to natural language understanding and genre categorization.

**Methodology:** - We implemented a BERT-based approach for genre prediction, specifically utilizing DistilBERT for its balance of performance and computational efficiency - The model was designed as a multi-label classifier to handle the reality that movies typically belong to multiple genres simultaneously - Our architecture consisted of a pre-trained DistilBERT base model followed by a custom classification head optimized for multi-label prediction - The classification layer utilized sigmoid activation functions to output independent probability scores for each of the 33 possible genres

### Decisions and Reasoning:

**Transformer Model Selection:** After evaluating several approaches, we selected DistilBERT over alternatives for the following reasons: - Performance-Efficiency Balance: DistilBERT offers 97% of BERT's performance while being 40% smaller and 60% faster, making it ideal for our deployment constraints - Transfer Learning Potential: Pre-trained on massive text corpora, DistilBERT already understands language patterns, requiring only fine-tuning for our specific task - Context Awareness: Unlike traditional bag-of-words approaches, transformer models understand word meanings in context, crucial for interpreting complex plot elements

- **Multi-Label Approach:** The multi-label architecture was essential since movies typically span multiple genres. Our analysis of the CMU dataset showed that movies belonged to an average of 2.4 genres each
- **Classification Head Design:** The custom classification head included:
  - A dropout layer (0.2) for regularization to prevent overfitting
  - A dense layer with 33 output neurons (one per genre) with sigmoid activation
  - Binary cross-entropy loss function specifically chosen for multi-label classification
- **Activation Function:** Sigmoid was selected over softmax because each genre prediction should be independent of others (a movie can be both “Action” and “Comedy” with high probability), which softmax doesn’t allow

**Methodology:** 1. The dataset was split into training (80%), validation (10%), and test (10%) sets using stratified sampling to maintain genre distribution 2. Genre labels were transformed using Multi-Hot Encoding, creating a binary vector of length 33 (the number of unique genres) 3. Text data was processed using DistilBERT’s tokenizer with a maximum sequence length of 256 tokens 4. The model was trained with the AdamW optimizer using a learning rate of  $3e-5$ , appropriate for fine-tuning transformer models 5. Early stopping was implemented to halt training when validation loss stopped improving, preventing overfitting 6. Checkpointing saved the best model version based on validation performance

**Decisions and Reasoning:** - **Data Split Ratios:** The 80/10/10 split was chosen to provide: - Sufficient training data to learn complex patterns across 33 genre categories - Adequate validation data for hyperparameter tuning and early stopping decisions - A substantial test set for final evaluation across multiple metrics

- **Tokenization Approach:** DistilBERT’s native tokenizer was essential to maintain compatibility with the pre-trained model’s vocabulary and embedding structure
- **Sequence Length Selection:** The 256 token limit was established after analyzing summary length distribution - capturing 95% of summaries completely while minimizing computational requirements
- **Optimization Strategy:** AdamW was selected over traditional Adam optimizer because weight decay regularization is particularly effective for fine-tuning large pre-trained models
- **Learning Rate:** The  $3e-5$  learning rate follows established best practices for transformer fine-tuning, balancing learning speed with stability

```

--- 6. Training ---
Starting training...
Epoch 1/5 300s 35ms/step - bin_accuracy: 0.8868 - f1_macro: 0.0396 - f1_micro: 0.1232 - f1_weighted: 0.0996 - loss: 0.3196 - val_bin_accuracy: 0.9198 - val_f1_macro: 0.0197 - val_f1_micro: 0.17
8225/8225
Epoch 2/5 315s 35ms/step - bin_accuracy: 0.9194 - f1_macro: 0.0276 - f1_micro: 0.1796 - f1_weighted: 0.1148 - loss: 0.2271 - val_bin_accuracy: 0.9211 - val_f1_macro: 0.0520 - val_f1_micro: 0.21
8225/8225
Epoch 3/5 331s 36ms/step - bin_accuracy: 0.9211 - f1_macro: 0.0710 - f1_micro: 0.2290 - f1_weighted: 0.1624 - loss: 0.2142 - val_bin_accuracy: 0.9229 - val_f1_macro: 0.1084 - val_f1_micro: 0.26
8225/8225
Epoch 4/5 312s 35ms/step - bin_accuracy: 0.9228 - f1_macro: 0.1277 - f1_micro: 0.2761 - f1_weighted: 0.2130 - loss: 0.2063 - val_bin_accuracy: 0.9243 - val_f1_macro: 0.1497 - val_f1_micro: 0.30
8225/8225
Epoch 5/5 335s 36ms/step - bin_accuracy: 0.9239 - f1_macro: 0.1595 - f1_micro: 0.3056 - f1_weighted: 0.2442 - loss: 0.2018 - val_bin_accuracy: 0.9253 - val_f1_macro: 0.1744 - val_f1_micro: 0.32
8225/8225
Training finished.
Best Validation F1 Micro: 0.3233

```

## Model Evaluation

Evaluating multi-label classification models requires specialized metrics beyond simple accuracy. Our comprehensive evaluation strategy assessed the model's performance across multiple dimensions.

**Methodology:** - The model was evaluated using metrics specifically suited for multi-label classification: - Hamming Loss: Measuring the fraction of incorrectly predicted labels - Micro F1-Score: Calculating precision and recall globally across all predictions - Macro F1-Score: Averaging F1 scores calculated independently for each genre - Weighted F1-Score: Similar to macro but weighted by genre frequency - Per-genre Precision, Recall, and F1 scores to understand performance variation across genres

- Confusion matrices were generated for each genre to visualize prediction patterns
- Thresholding analysis was performed to optimize the probability cutoff for genre assignment

**Decisions and Reasoning: - Metric Selection:** Traditional accuracy is inadequate for multi-label classification because: - It doesn't account for partial correctness (getting some genres right but not others) - It fails to address class imbalance (some genres are much more common than others) - It doesn't differentiate between different types of errors

- **Threshold Optimization:** After experimentation with various thresholds (0.3, 0.4, 0.5, 0.6), we selected 0.5 as the optimal balance between precision and recall. Lower thresholds increased recall but at significant cost to precision, while higher thresholds did the opposite
- **Error Analysis Focus:** We particularly analyzed confusion patterns for underrepresented genres, discovering that:
  - Certain genre pairs were frequently confused (e.g., "Thriller" and "Horror")
  - Very rare genres (e.g., "Silent film") had lower performance due to limited training examples
  - Some genres were inherently more difficult to predict from text alone (e.g., "Black and white")

```

--- 7. Evaluation ---
Loading best model for evaluation...
129/129 ████████████████████ 36s 257ms/step

Test Set Evaluation Metrics:
Exact Match Ratio (Accuracy): 0.1062
Hamming Loss: 0.0734
F1 Score (Micro): 0.3875
F1 Score (Macro): 0.2459
F1 Score (Weighted): 0.3308

Classification Report (Test Set):

```

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Action          | 0.59      | 0.25   | 0.35     | 605     |
| ActionAdventure | 0.53      | 0.11   | 0.18     | 368     |
| Adventure       | 0.71      | 0.13   | 0.23     | 326     |
| Animation       | 0.70      | 0.42   | 0.53     | 236     |
| Blackandwhite   | 0.52      | 0.04   | 0.07     | 372     |
| Bollywood       | 0.00      | 0.00   | 0.00     | 94      |
| Comedy          | 0.67      | 0.33   | 0.44     | 1059    |
| Comedy-drama    | 0.00      | 0.00   | 0.00     | 145     |
| Comedyfilm      | 0.00      | 0.00   | 0.00     | 185     |
| CrimeFiction    | 0.65      | 0.21   | 0.32     | 431     |
| ...             |           |        |          |         |
| macro avg       | 0.52      | 0.19   | 0.25     | 11530   |
| weighted avg    | 0.57      | 0.27   | 0.33     | 11530   |
| samples avg     | 0.53      | 0.33   | 0.37     | 11530   |

## 6. Interactive User Interface

### GUI Design and Implementation

The technical capabilities of Filmception are made accessible through a thoughtfully designed graphical user interface that prioritizes user experience, accessibility, and visual appeal.

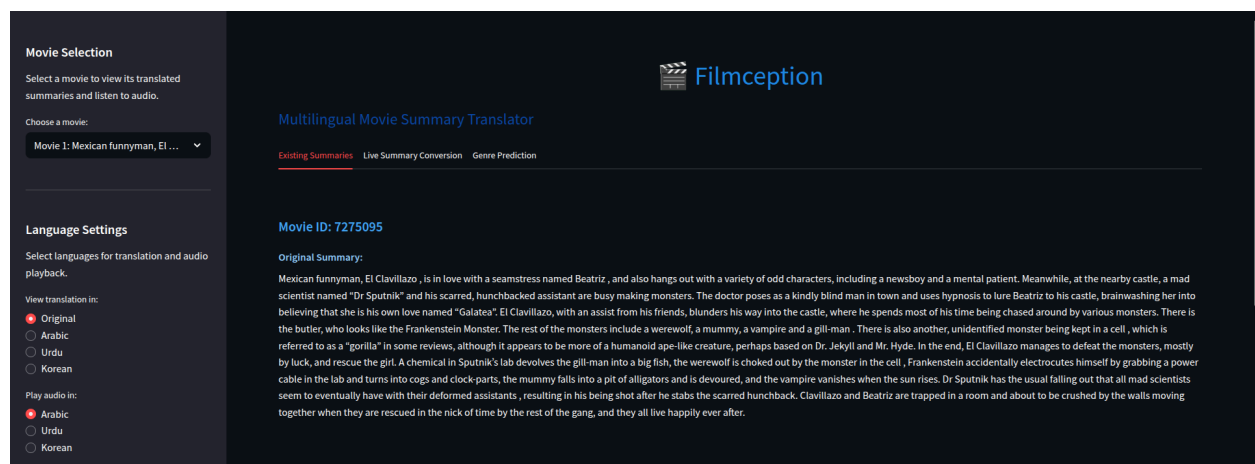
**Methodology:** - We developed a Streamlit-based web interface that integrates all system components: - Text input for movie summaries with clear instruction and example text - Language selection for translation and audio generation - Genre prediction visualization with confidence scores - Audio playback with intuitive controls - Clear visual feedback for processing stages

- Custom CSS styling was implemented to enhance visual appeal and usability
- The interface was designed with a responsive layout that adapts to different screen sizes
- Caching mechanisms were implemented for resource-intensive operations



**Decisions and Reasoning: - Framework Selection:** Streamlit was chosen as our UI framework after evaluating several alternatives (Flask, Dash, Gradio) for these specific advantages: - Rapid Development: Streamlit's design philosophy allowed us to create a functional UI with minimal code - Python Integration: Seamless integration with our existing Python-based backend components - Interactive Elements: Built-in support for the widgets and controls our application required - Deployment Simplicity: Straightforward path to deployment without complex configuration

- **Layout Design:** The interface follows a logical flow matching the user's mental model:
  1. Input section at the top (what the user needs to provide)
  2. Processing options in the middle (what the user wants to do with their input)
  3. Results section at the bottom (what the system produces)
- **Visual Design Elements:** Custom styling was implemented to enhance usability and engagement:
  - Color scheme based on blue tones to convey trust and professionalism
  - Clear typography hierarchy to distinguish between different information types
  - Adequate whitespace and separation between functional areas
  - Consistent styling of interactive elements for predictable user experience
- **Error Handling:** The interface incorporates robust error handling with user-friendly messages for common issues:
  - Empty input validation with helpful prompts
  - Graceful handling of translation or audio generation failures
  - Clear loading indicators during processing operations



Movie Selection

Select a movie to view its translated summaries and listen to audio.

Choose a movie:

Movie 1: Mexican funnyman, El ...

Language Settings

Select languages for translation and audio playback.

View translation in:

Original

Arabic

Urdu

Korean

Play audio in:

Arabic

Urdu

Korean

About Filmception

Filmception is an AI-powered multilingual movie summary translator and genre classifier built for the Artificial Intelligence

0:00 / 1:39

Additional Features

Compare All Translations

Arabic

Urdu

Korean

Compare All Audio Files

Arabic Audio

Urdu Audio

Korean Audio

Movie Selection

Select a movie to view its translated summaries and listen to audio.

Choose a movie:

Movie 1: Mexican funnyman, El ...

Language Settings

Select languages for translation and audio playback.

View translation in:

Original

Arabic

Urdu

Korean

Play audio in:

Arabic

Urdu

Korean

About Filmception

Filmception is an AI-powered multilingual movie summary translator and genre classifier built for the Artificial Intelligence

Filmception

Multilingual Movie Summary Translator

Existing SummariesLive Summary ConversionGenre Prediction

Live Summary Conversion

Enter a movie summary and convert it to translated text and audio in the selected language.

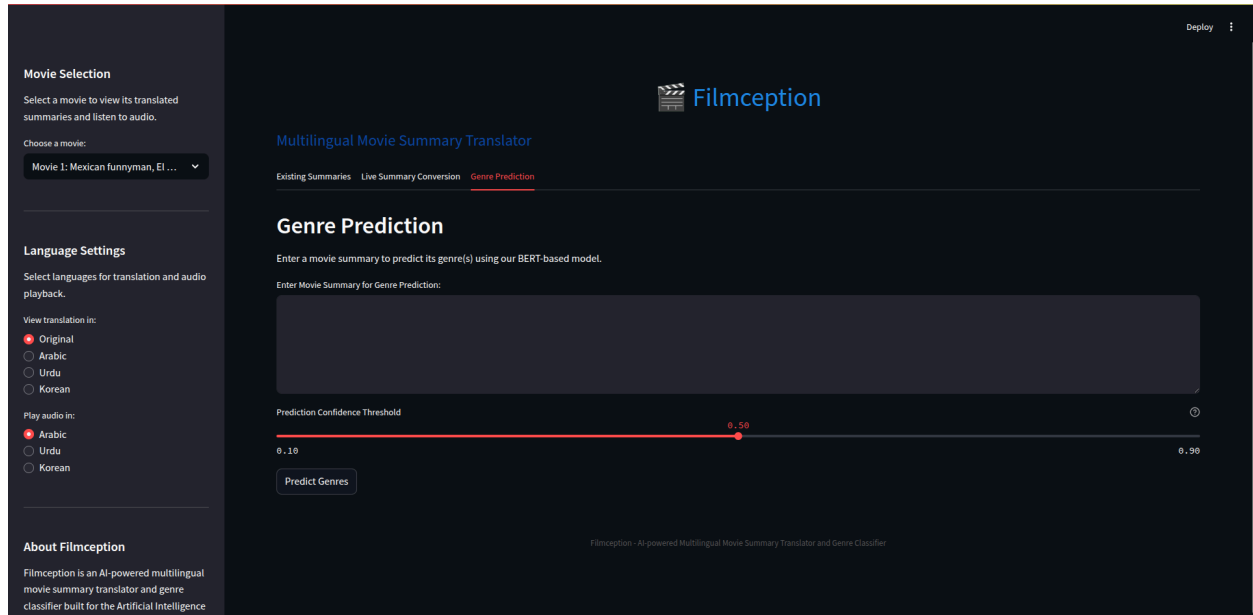
Enter Movie Summary:

Select Conversion Language:

English

Convert Summary

Filmception - AI-powered Multilingual Movie Summary Translator and Genre Classifier



## 7. Integration and Workflow

### System Architecture

The complete Filmception system integrates multiple specialized components into a cohesive whole, requiring careful architectural planning to ensure efficient data flow and component interaction.

**Methodology:** - The system architecture follows a modular design with five primary components:

1. **Data Processing Module:** Handles loading, cleaning, and preparing movie data
  2. **Translation Module:** Manages the conversion of text between languages
  3. **Audio Generation Module:** Transforms text into spoken audio
  4. **Genre Prediction Module:** Implements the BERT-based classification model
  5. **User Interface Module:** Provides the interactive front-end experience
- Each module is designed with clear input/output interfaces for seamless integration
  - Data flows through the system in a defined pipeline with appropriate transformation at each stage
  - Caching mechanisms minimize redundant computation for repeated operations

### Decisions and Reasoning:

**Modular Design Philosophy:** The separation of concerns into distinct modules was implemented for several key benefits: - Independent Development: Team members could work on different modules concurrently - Isolated Testing: Each module could be tested separately before integration - Maintainability: Changes to one module minimally impact

others - Reusability: Components like the translation module could potentially be reused in other projects

- **Data Flow Optimization:** Careful attention was paid to efficient data passing between components:
  - Intermediate results are cached when appropriate to avoid redundant processing
  - Batch processing is used where beneficial (especially for API calls)
  - Memory usage is managed by clearing intermediate results when no longer needed
- **Resource Management:** The architecture incorporates strategies to handle resource-intensive operations:
  - The BERT model is loaded once and reused for multiple predictions
  - Translation requests are rate-limited to prevent API issues
  - Audio files are generated on demand and cached for reuse
- **Error Handling Strategy:** The architecture implements a comprehensive error handling approach:
  - Each module catches and manages its own specific errors
  - Errors are logged with appropriate context for debugging
  - User-facing error messages are clear and actionable
  - System stability is maintained even when individual operations fail

## 8. Challenges and Solutions

### Translation and Language Support

**Challenge:** Supporting diverse languages including right-to-left scripts (Arabic) and non-Latin alphabets (Korean) presented significant technical hurdles in text processing, display, and audio conversion.

**Solution:** We implemented comprehensive Unicode handling throughout the system, ensuring proper text encoding and display. For right-to-left languages, we applied appropriate text direction controls in the UI. Font selection was carefully managed to ensure all characters rendered correctly across languages. Additionally, we incorporated language-specific text preprocessing to address unique linguistic features before translation and audio generation.

### Model Performance and Efficiency

**Challenge:** Balancing the predictive accuracy of our genre classification model with reasonable computational requirements proved difficult, especially considering the need for real-time predictions in the user interface.

**Solution:** We selected DistilBERT as our base model after benchmarking several alternatives (BERT, RoBERTa, XLNet) for the optimal balance of accuracy and efficiency. Implementation of model caching ensured the model was loaded only once. We also

optimized batch processing for prediction and implemented asynchronous processing where possible to maintain UI responsiveness even during complex predictions. Performance profiling identified and eliminated computational bottlenecks in the prediction pipeline.

### Multi-Label Classification Complexity

**Challenge:** The multi-label nature of movie genre classification introduced complexities in model training, evaluation, and threshold selection, particularly given the significant imbalance across genres (some genres had 50x more examples than others).

**Solution:** We implemented a comprehensive approach to address these challenges. Class weighting was applied during training to compensate for genre imbalance. We used specialized metrics (Macro F1, Hamming Loss) that appropriately reflect performance in multi-label scenarios. Genre-specific thresholds were implemented based on precision-recall analysis for each category. Additionally, we incorporated data augmentation techniques to increase representation of minority genres during training.

### Audio Generation Quality

**Challenge:** Generating natural-sounding audio across multiple languages while managing API limitations and handling edge cases (very long texts, unusual characters) proved technically challenging.

**Solution:** We implemented a robust audio generation pipeline with several optimizations. Text segmentation was applied for very long summaries, breaking them into manageable chunks. Language-specific preprocessing removed or normalized characters that caused issues in the TTS engine. A comprehensive error handling system managed TTS failures gracefully, with fallback options when primary generation failed. We also implemented adaptive rate limiting based on API response codes to maximize throughput while preventing errors.

## 9. Conclusion

### Project Achievements

Filmception successfully demonstrates the power of combining multiple AI capabilities into a cohesive system that provides genuine value to users. Our key achievements include:

- Development of a complete end-to-end system for movie summary analysis, translation, and genre prediction
- Implementation of a multi-language translation and audio generation pipeline supporting diverse scripts and language families
- Creation of an effective multi-label classification model for movie genre prediction using state-of-the-art transformer architecture
- Design and implementation of an intuitive, accessible user interface that makes complex AI functionality approachable

- Integration of multiple components into a cohesive system with efficient data flow and error handling

The project demonstrates practical applications of natural language processing techniques, showcasing how AI can make content more accessible across language barriers while extracting meaningful insights from text.

## 10. References

- CMU Movie Summary Dataset:  
<https://www.kaggle.com/datasets/msafi04/movies-genre-dataset-cmu-movie-summary>
- DistilBERT: [https://huggingface.co/docs/transformers/model\\_doc/distilbert](https://huggingface.co/docs/transformers/model_doc/distilbert)
- Streamlit Documentation: <https://docs.streamlit.io>
- Google Translate API Documentation: <https://cloud.google.com/translate/docs>
- Google Text-to-Speech API Documentation:  
<https://cloud.google.com/text-to-speech/docs>
- TensorFlow Documentation: [https://www.tensorflow.org/api\\_docs](https://www.tensorflow.org/api_docs)
- Hugging Face Transformers: <https://huggingface.co/docs/transformers/index>
- Multi-label Classification Metrics:  
[https://scikit-learn.org/stable/modules/model\\_evaluation.html#multilabel-ranking-metrics](https://scikit-learn.org/stable/modules/model_evaluation.html#multilabel-ranking-metrics)